



Chapter 15. *Atomic Variables and Nonblocking Synchronization*

Many of the classes in `java.util.concurrent`, such as `Semaphore` and `ConcurrentLinkedQueue`, provide better performance and scalability than alternatives using `synchronized`. In this chapter, we take a look at the primary source of this performance boost: atomic variables and nonblocking synchronization.

Much of the recent research on concurrent algorithms has focused on *nonblocking algorithms*, which use low-level atomic machine instructions such as *compare-and-swap* instead of locks to ensure data integrity under concurrent access. Nonblocking algorithms are used extensively in operating systems and JVMs for thread and process scheduling, garbage collection, and to implement locks and other concurrent data structures.

Nonblocking algorithms are considerably more complicated to design and implement than lock-based alternatives, but they can offer significant scalability and liveness advantages. They coordinate at a finer level of granularity and can greatly reduce scheduling overhead because they don't block when multiple threads contend for the same data. Further, they are immune to deadlock and other liveness problems. In lock-based algorithms, other threads cannot make progress if a thread goes to sleep or spins while holding a lock, whereas nonblocking algorithms are impervious to individual thread failures. As of Java 5.0, it is possible to build efficient nonblocking algorithms in Java using the *atomic variable classes* such as `AtomicInteger` and `AtomicReference`.

Atomic variables can also be used as “better volatile variables” even if you are not developing nonblocking algorithms. Atomic variables offer the same memory semantics as volatile variables, but with additional support for atomic updates—making them ideal for counters, sequence

generators, and statistics gathering while offering better scalability than lock-based alternatives.

15.1. Disadvantages of Locking

Coordinating access to shared state using a consistent locking protocol ensures that whichever thread holds the lock guarding a set of variables has exclusive access to those variables, and that any changes made to those variables are visible to other threads that subsequently acquire the lock.

Modern JVMs can optimize uncontended lock acquisition and release fairly effectively, but if multiple threads request the lock at the same time the JVM enlists the help of the operating system. If it gets to this point, some unfortunate thread will be suspended and have to be resumed later.^[1] When that thread is resumed, it may have to wait for other threads to finish their scheduling quanta before it is actually scheduled. Suspending and resuming a thread has a lot of overhead and generally entails a lengthy interruption. For lock-based classes with fine-grained operations (such as the synchronized collections classes, where most methods contain only a few operations), the ratio of scheduling overhead to useful work can be quite high *when the lock is frequently contended*.

[1] A smart JVM need not necessarily suspend a thread if it contends for a lock; it could use profiling data to decide adaptively between suspension and spin locking based on how long the lock has been held during previous acquisitions.

Volatile variables are a lighter-weight synchronization mechanism than locking because they do not involve context switches or thread scheduling. However, volatile variables have some limitations compared to locking: while they provide similar visibility guarantees, they cannot be used to construct atomic compound actions. This means that volatile variables cannot be used when one variable depends on another, or when the new value of a variable depends on its old value. This limits when volatile variables are appropriate, since they cannot be used to reliably implement common tools such as counters or mutexes.^[2]

[2] It is theoretically possible, though wholly impractical, to use the semantics of `volatile` to construct mutexes and other synchronizers; see ([Raynal, 1986](#)).

For example, while the increment operation (`++i`) may *look* like an atomic operation, it is actually three distinct operations—fetch the current value of the variable, add one to it, and then write the updated value back. In order to not lose an update, the entire read-modify-write operation must be atomic. So far, the only way we've seen to do this is with locking, as in `Counter` on page [56](#).

`Counter` is thread-safe, and in the presence of little or no contention performs just fine. But under contention, performance suffers because of context-switch overhead and scheduling delays. When locks are held so briefly, being put to sleep is a harsh penalty for asking for the lock at the wrong time.

Locking has a few other disadvantages. When a thread is waiting for a lock, it cannot do anything else. If a thread holding a lock is delayed (due to a page fault, scheduling delay, or the like), then no thread that needs that lock can make progress. This can be a serious problem if the blocked thread is a high-priority thread but the thread holding the lock is a lower-priority thread—a performance hazard known as *priority inversion*. Even though the higher-priority thread should have precedence, it must wait until the lock is released, and this effectively downgrades its priority to that of the lower-priority thread. If a thread holding a lock is permanently blocked (due to an infinite loop, deadlock, livelock, or other liveness failure), any threads waiting for that lock can never make progress.

Even ignoring these hazards, locking is simply a heavyweight mechanism for fine-grained operations such as incrementing a counter. It would be nice to have a finer-grained technique for managing contention between threads—something like volatile variables, but offering the possibility of atomic updates as well. Happily, modern processors offer us precisely such a mechanism.

15.2. Hardware Support for Concurrency

Exclusive locking is a *pessimistic* technique—it assumes the worst (if you don't lock your door, gremlins will come in and rearrange your stuff) and doesn't proceed until you can guarantee, by acquiring the appropriate locks, that other threads will not interfere.

For fine-grained operations, there is an alternate approach that is often more efficient—the *optimistic* approach, whereby you proceed with an update, hopeful that you can complete it without interference. This approach relies on *collision detection* to determine if there has been interference from other parties during the update, in which case the operation fails and can be retried (or not). The optimistic approach is like the old saying, “It is easier to obtain forgiveness than permission”, where “easier” here means “more efficient”.

Processors designed for multiprocessor operation provide special instructions for managing concurrent access to shared variables. Early processors had atomic *test-and-set*, *fetch-and-increment*, or *swap* instructions sufficient for implementing mutexes that could in turn be used to implement more sophisticated concurrent objects. Today, nearly every modern processor has some form of atomic read-modify-write instruction, such as *compare-and-swap* or *load-linked/store-conditional*. Operating systems and JVMs use these instructions to implement locks and concurrent data structures, but until Java 5.0 they had not been available directly to Java classes.

15.2.1. Compare and Swap

The approach taken by most processor architectures, including IA32 and Sparc, is to implement a *compare-and-swap* (CAS) instruction. (Other processors, such as PowerPC, implement the same functionality with a pair of instructions: *loadlinked* and *store-conditional*.) CAS has three operands—a memory location *V* on which to operate, the expected old value *A*, and the newvalue *B*. CAS atomically updates *V* to the new value *B*, but only if the value in *V* matches the expected old value *A*; otherwise it does nothing. In either case, it returns the value currently in *V*. (The variant called compare-and-set instead returns whether the operation succeeded.) CAS means “I think *V* should have the value *A*; if it does, put *B* there, otherwise don't change it but tell me I was wrong.” CAS is an optimistic tech-

nique—it proceeds with the update in the hope of success, and can detect failure if another thread has updated the variable since it was last examined. `SimulatedCAS` in [Listing 15.1](#) illustrates the semantics (but not the implementation or performance) of CAS.

When multiple threads attempt to update the same variable simultaneously using CAS, one wins and updates the variable's value, and the rest lose. But the losers are not punished by suspension, as they could be if they failed to acquire a lock; instead, they are told that they didn't win the race this time but can try again. Because a thread that loses a CAS is not blocked, it can decide whether it wants to try again, take some other recovery action, or do nothing.^[3] This flexibility eliminates many of the liveness hazards associated with locking (though in unusual cases can introduce the risk of *livelock*—see [Section 10.3.3](#)).

[3] Doing nothing may be a perfectly sensible response to a failed CAS; in some nonblocking algorithms, such as the linked queue algorithm in [Section 15.4.2](#), a failed CAS means that someone else already did the work you were planning to do.

Listing 15.1. Simulated CAS Operation.

```
@ThreadSafe
public class SimulatedCAS {
    @GuardedBy("this") private int value;

    public synchronized int get() { return value; }

    public synchronized int compareAndSwap(int expectedValue,
                                           int newValue) {
        int oldValue = value;
        if (oldValue == expectedValue)
            value = newValue;
        return oldValue;
    }

    public synchronized boolean compareAndSet(int expectedValue,
                                              int newValue) {
        return (expectedValue
                == compareAndSwap(expectedValue, newValue));
    }
}
```

The typical pattern for using CAS is first to read the value A from V , derive the new value B from A , and then use CAS to atomically change V from A to B so long as no other thread has changed V to another value in the meantime. CAS addresses the problem of implementing atomic read-modify-write sequences without locking, because it can detect interference from other threads.

15.2.2. A Nonblocking Counter

`CasCounter` in [Listing 15.2](#) implements a thread-safe counter using CAS. The increment operation follows the canonical form—fetch the old value, transform it to the new value (adding one), and use CAS to set the new value. If the CAS fails, the operation is immediately retried. Retrying repeatedly is usually a reasonable strategy, although in cases of extreme contention it might be desirable to wait or back off before retrying to avoid livelock.

`CasCounter` does not block, though it may have to retry several^[4] times if other threads are updating the counter at the same time. (In practice, if all you need is a counter or sequence generator, just use `AtomicInteger` or `AtomicLong`, which provide atomic increment and other arithmetic methods.)

^[4] Theoretically, it could have to retry arbitrarily many times if other threads keep winning the CAS race; in practice, this sort of starvation rarely happens.

Listing 15.2. Nonblocking Counter Using CAS.


```
@ThreadSafe
public class CasCounter {
    private SimulatedCAS value;

    public int getValue() {
        return value.get();
    }

    public int increment() {
        int v;
        do {
            v = value.get();
        }
        while (v != value.compareAndSwap(v, v + 1));
        return v + 1;
    }
}
```

At first glance, the CAS-based counter looks as if it should perform worse than a lock-based counter; it has more operations and a more complicated control flow, and depends on the seemingly complicated CAS operation. But in reality, CAS-based counters significantly outperform lock-based counters if there is even a small amount of contention, and often even if there is no contention. The fast path for uncontended lock acquisition typically requires at least one CAS plus other lock-related housekeeping, so more work is going on in the best case for a lock-based counter than in the normal case for the CAS-based counter. Since the CAS succeeds most of the time (assuming low to moderate contention), the hardware will correctly predict the branch implicit in the `while` loop, minimizing the overhead of the more complicated control logic.

The language syntax for locking may be compact, but the work done by the JVM and OS to manage locks is not. Locking entails traversing a relatively complicated code path in the JVM and may entail OS-level locking, thread suspension, and context switches. In the best case, locking requires at least one CAS, so using locks moves the CAS out of sight but doesn't save any actual execution cost. On the other hand, executing a CAS from within the program involves no JVM code, system calls, or scheduling activity. What looks like a longer code path at the application level is in fact a much shorter code path when JVM and OS activity are taken into account. The primary disadvantage of CAS is that it forces the caller to deal with contention (by retrying, backing off, or giving up),

whereas locks deal with contention automatically by blocking until the lock is available.^[5]

^[5] Actually, the biggest disadvantage of CAS is the difficulty of constructing the surrounding algorithms correctly.

CAS performance varies widely across processors. On a single-CPU system, a CAS typically takes on the order of a handful of clock cycles, since no synchronization across processors is necessary. As of this writing, the cost of an uncontended CAS on multiple CPU systems ranges from about ten to about 150 cycles; CAS performance is a rapidly moving target and varies not only across architectures but even across versions of the same processor. Competitive forces will likely result in continued CAS performance improvement over the next several years. A good rule of thumb is that the cost of the “fast path” for *uncontended* lock acquisition and release on most processors is approximately twice the cost of a CAS.

15.2.3. CAS Support in the JVM

So, how does Java code convince the processor to execute a CAS on its behalf? Prior to Java 5.0, there was no way to do this short of writing native code. In Java 5.0, low-level support was added to expose CAS operations on `int`, `long`, and object references, and the JVM compiles these into the most efficient means provided by the underlying hardware. On platforms supporting CAS, the runtime inlines them into the appropriate machine instruction(s); in the worst case, if a CAS-like instruction is not available the JVM uses a spin lock. This low-level JVM support is used by the atomic variable classes (`AtomicXxx` in `java.util.concurrent.atomic`) to provide an efficient CAS operation on numeric and reference types; these atomic variable classes are used, directly or indirectly, to implement most of the classes in `java.util.concurrent`.

15.3. Atomic Variable Classes

Atomic variables are finer-grained and lighter-weight than locks, and are critical for implementing high-performance concurrent code on multiprocessor systems. Atomic variables limit the scope of contention to a single variable; this is as finegrained as you can get (assuming your algorithm

can even be implemented using such fine granularity). The fast (uncontended) path for updating an atomic variable is no slower than the fast path for acquiring a lock, and usually faster; the slow path is definitely faster than the slow path for locks because it does not involve suspending and rescheduling threads. With algorithms based on atomic variables instead of locks, threads are more likely to be able to proceed without delay and have an easier time recovering if they do experience contention.

The atomic variable classes provide a generalization of volatile variables to support atomic conditional read-modify-write operations.

`AtomicInteger` represents an `int` value, and provides `get` and `set` methods with the same memory semantics as reads and writes to a volatile `int`. It also provides an atomic `compareAndSet` method (which if successful has the memory effects of both reading and writing a volatile variable) and, for convenience, atomic add, increment, and decrement methods. `AtomicInteger` bears a superficial resemblance to an extended `Counter` class, but offers far greater scalability under contention because it can directly exploit underlying hardware support for concurrency.

There are twelve atomic variable classes, divided into four groups: scalars, field updaters, arrays, and compound variables. The most commonly used atomic variables are the scalars: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, and `AtomicReference`. All support CAS; the `Integer` and `Long` versions support arithmetic as well. (To simulate atomic variables of other primitive types, you can cast `short` or `byte` values to and from `int`, and use `floatToIntBits` or `doubleToLongBits` for floating-point numbers.)

The atomic array classes (available in `Integer`, `Long`, and `Reference` versions) are arrays whose elements can be updated atomically. The atomic array classes provide volatile access semantics to the elements of the array, a feature not available for ordinary arrays—a volatile array has volatile semantics only for the array reference, not for its elements. (The other types of atomic variables are discussed in [Sections 15.4.3](#) and [15.4.4](#).)

While the atomic scalar classes extend `Number`, they do not extend the primitive wrapper classes such as `Integer` or `Long`. In fact, they cannot: the primitive wrapper classes are immutable whereas the atomic variable classes are mutable. The atomic variable classes also do not redefine `hashCode` or `equals`; each instance is distinct. Like most mutable objects, they are not good candidates for keys in hash-based collections.

15.3.1. Atomics as “Better Volatiles”

In [Section 3.4.2](#), we used a `volatile` reference to an immutable object to update multiple state variables atomically. That example relied on check-then-act, but in that particular case the race was harmless because we did not care if we occasionally lost an update. In most other situations, such a check-then-act would not be harmless and could compromise data integrity. For example, `NumberRange` on page [67](#) could not be implemented safely with a `volatile` reference to an immutable holder object for the upper and lower bounds, nor with using atomic integers to store the bounds. Because an invariant constrains the two numbers and they cannot be updated simultaneously while preserving the invariant, a number range class using `volatile` references or multiple atomic integers will have unsafe check-then-act sequences.

We can combine the technique from `OneValueCache` with atomic references to close the race condition by *atomically* updating the reference to an immutable object holding the lower and upper bounds.

`CasNumberRange` in [Listing 15.3](#) uses an `AtomicReference` to an `IntPair` to hold the state; by using `compareAndSet` it can update the upper or lower bound without the race conditions of `NumberRange`.

Listing 15.3. Preserving Multivariable Invariants Using CAS.

```

public class CasNumberRange {
    @Immutable
    private static class IntPair {
        final int lower; // Invariant: lower <= upper
        final int upper;
        ...
    }
    private final AtomicReference<IntPair> values =
        new AtomicReference<IntPair>(new IntPair(0, 0));

    public int getLower() { return values.get().lower; }
    public int getUpper() { return values.get().upper; }

    public void setLower(int i) {
        while (true) {
            IntPair oldv = values.get();
            if (i > oldv.upper)
                throw new IllegalArgumentException(
                    "Can't set lower to " + i + " > upper");
            IntPair newv = new IntPair(i, oldv.upper);
            if (values.compareAndSet(oldv, newv))
                return;
        }
    }
    // similarly for setUpper
}

```

15.3.2. Performance Comparison: Locks Versus Atomic Variables

To demonstrate the differences in scalability between locks and atomic variables, we constructed a benchmark comparing several implementations of a pseudorandom number generator (PRNG). In a PRNG, the next “random” number is a deterministic function of the previous number, so a PRNG must remember the previous number as part of its state.

[Listings 15.4](#) and [15.5](#) show two implementations of a thread-safe PRNG, one using `ReentrantLock` and the other using `AtomicInteger`. The test driver invokes each repeatedly; each iteration generates a random number (which fetches and modifies the shared `seed` state) and also performs a number of “busy-work” iterations that operate strictly on thread-local data. This simulates typical operations that include some portion of operating on shared state and some portion of operating on thread-local state.

Figures 15.1 and **15.2** show throughput with low and moderate levels of simulated work in each iteration. With a low level of thread-local computation, the lock or atomic variable experiences heavy contention; with more thread-local computation, the lock or atomic variable experiences less contention since it is accessed less often by each thread.

Figure 15.1. Lock and AtomicInteger Performance Under High Contention.

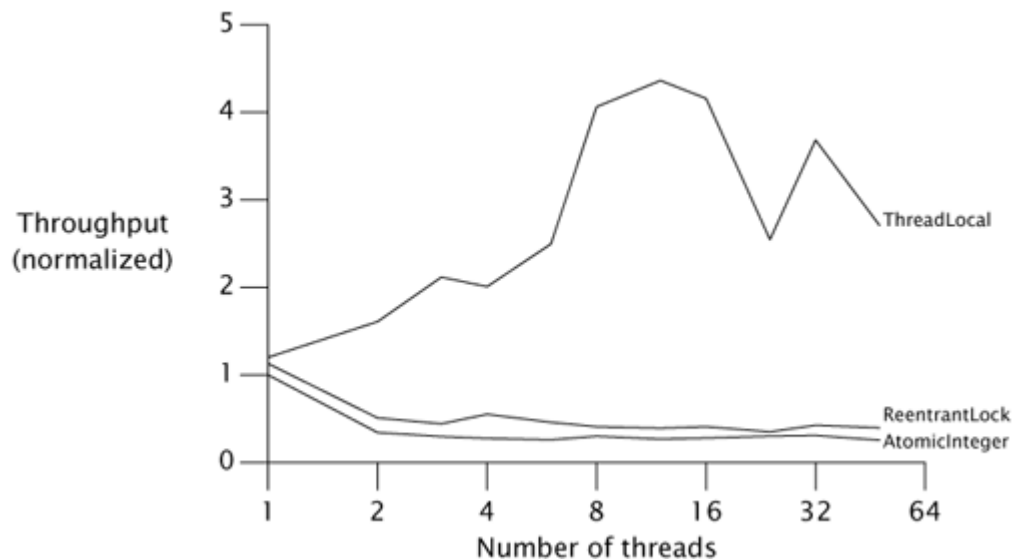
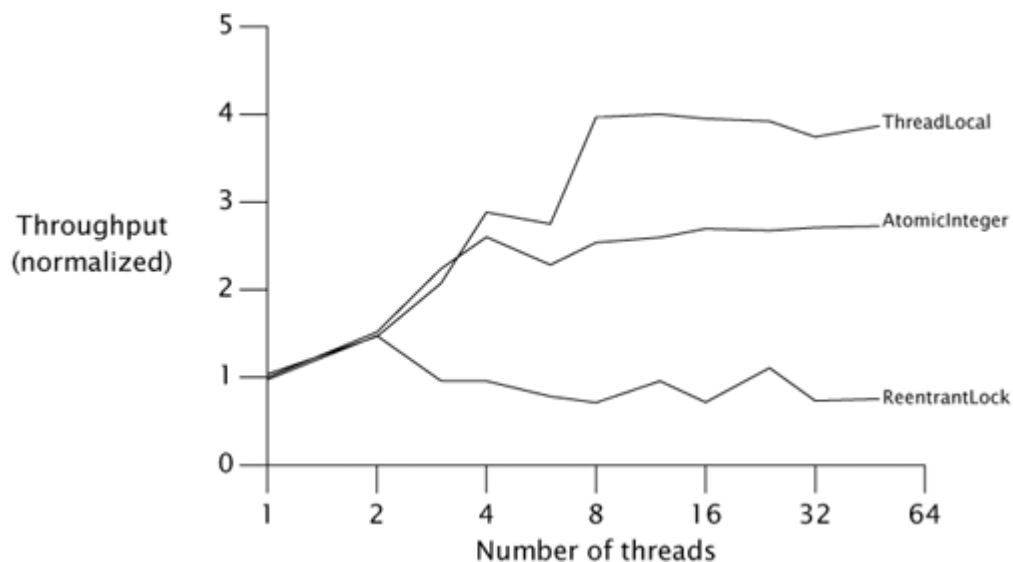


Figure 15.2. Lock and AtomicInteger Performance Under Moderate Contention.



Listing 15.4. Random Number Generator Using ReentrantLock.

```

@ThreadSafe
public class ReentrantLockPseudoRandom extends PseudoRandom {
    private final Lock lock = new ReentrantLock(false);
    private int seed;

    ReentrantLockPseudoRandom(int seed) {
        this.seed = seed;
    }

    public int nextInt(int n) {
        lock.lock();
        try {
            int s = seed;
            seed = calculateNext(s);
            int remainder = s % n;
            return remainder > 0 ? remainder : remainder + n;
        } finally {
            lock.unlock();
        }
    }
}

```

Listing 15.5. Random Number Generator Using `AtomicInteger`.

```

@ThreadSafe
public class AtomicPseudoRandom extends PseudoRandom {
    private AtomicInteger seed;

    AtomicPseudoRandom(int seed) {
        this.seed = new AtomicInteger(seed);
    }

    public int nextInt(int n) {
        while (true) {
            int s = seed.get();
            int nextSeed = calculateNext(s);
            if (seed.compareAndSet(s, nextSeed)) {
                int remainder = s % n;
                return remainder > 0 ? remainder : remainder + n;
            }
        }
    }
}

```

As these graphs show, at high contention levels locking tends to outperform atomic variables, but at more realistic contention levels atomic variables outperform locks.^[6] This is because a lock reacts to contention by suspending threads, reducing CPU usage and synchronization traffic on the shared memory bus. (This is similar to how blocking producers in a

producer-consumer design reduces the load on consumers and thereby lets them catch up.) On the other hand, with atomic variables, contention management is pushed back to the calling class. Like most CAS-based algorithms, `AtomicPseudoRandom` reacts to contention by trying again immediately, which is usually the right approach but in a high-contention environment just creates more contention.

[6] The same holds true in other domains: traffic lights provide better throughput for high traffic but rotaries provide better throughput for low traffic; the contention scheme used by ethernet networks performs better at low traffic levels, but the token-passing scheme used by token ring networks does better with heavy traffic.

Before we condemn `AtomicPseudoRandom` as poorly written or atomic variables as a poor choice compared to locks, we should realize that the level of contention in **Figure 15.1** is unrealistically high: no real program does nothing but contend for a lock or atomic variable. In practice, atomics tend to scale better than locks because atomics deal more effectively with typical contention levels.

The performance reversal between locks and atomics at differing levels of contention illustrates the strengths and weaknesses of each. With low to moderate contention, atomics offer better scalability; with high contention, locks offer better contention avoidance. (CAS-based algorithms also outperform lock-based ones on single-CPU systems, since a CAS always succeeds on a single-CPU system except in the unlikely case that a thread is preempted in the middle of the read-modify-write operation.)

Figures 15.1 and **15.2** include a third curve; an implementation of `PseudoRandom` that uses a `ThreadLocal` for the PRNG state. This implementation approach changes the behavior of the class—each thread sees its own private sequence of pseudorandom numbers, instead of all threads sharing one sequence—but illustrates that it is often cheaper to not share state at all if it can be avoided. We can improve scalability by dealing more effectively with contention, but true scalability is achieved only by eliminating contention entirely.

15.4. Nonblocking Algorithms

Lock-based algorithms are at risk for a number of liveness failures. If a thread holding a lock is delayed due to blocking I/O, page fault, or other delay, it is possible that no thread will make progress. An algorithm is called *nonblocking* if failure or suspension of any thread cannot cause failure or suspension of another thread; an algorithm is called *lock-free* if, at each step, *some* thread can make progress. Algorithms that use CAS exclusively for coordination between threads can, if constructed correctly, be both nonblocking and lock-free. An uncontended CAS always succeeds, and if multiple threads contend for a CAS, one always wins and therefore makes progress. Nonblocking algorithms are also immune to deadlock or priority inversion (though they can exhibit starvation or livelock because they can involve repeated retries). We've seen one nonblocking algorithm so far: `CasCounter`. Good nonblocking algorithms are known for many common data structures, including stacks, queues, priority queues, and hash tables—though designing new ones is a task best left to experts.

15.4.1. A Nonblocking Stack

Nonblocking algorithms are considerably more complicated than their lock-based equivalents. The key to creating nonblocking algorithms is figuring out how to limit the scope of atomic changes to a *single* variable while maintaining data consistency. In linked collection classes such as queues, you can sometimes get away with expressing state transformations as changes to individual links and using an `AtomicReference` to represent each link that must be updated atomically.

Stacks are the simplest linked data structure: each element refers to only one other element and each element is referred to by only one object reference. `ConcurrentStack` in [Listing 15.6](#) shows how to construct a stack using atomic references. The stack is a linked list of `Node` elements, rooted at `top`, each of which contains a value and a link to the next element. The `push` method prepares a new link node whose `next` field refers to the current top of the stack, and then uses CAS to try to install it on the top of the stack. If the same node is still on the top of the stack as when we started, the CAS succeeds; if the top node has changed (because another thread has added or removed elements since we started), the CAS fails and `push` updates the new node based on the current stack state

and tries again. In either case, the stack is still in a consistent state after the CAS.

`CasCounter` and `ConcurrentStack` illustrate characteristics of all non-blocking algorithms: some work is done speculatively and may have to be redone. In `ConcurrentStack`, when we construct the `Node` representing the new element, we are hoping that the value of the `next` reference will still be correct by the time it is installed on the stack, but are prepared to retry in the event of contention.

Nonblocking algorithms like `ConcurrentStack` derive their thread safety from the fact that, like locking, `compareAndSet` provides both atomicity and visibility guarantees. When a thread changes the state of the stack, it does so with a `compareAndSet`, which has the memory effects of a volatile write. When a thread examines the stack, it does so by calling `get` on the same `AtomicReference`, which has the memory effects of a volatile read. So any changes made by one thread are safely published to any other thread that examines the state of the list. And the list is modified with a `compareAndSet` that atomically either updates the `top` reference or fails if it detects interference from another thread.

15.4.2. A Nonblocking Linked List

The two nonblocking algorithms we've seen so far, the counter and the stack, illustrate the basic pattern of using CAS to update a value speculatively, retrying if the update fails. The trick to building nonblocking algorithms is to limit the scope of atomic changes to a single variable. With counters this is trivial, and with a stack it is straightforward enough, but for more complicated data structures such as queues, hash tables, or trees, it can get a lot trickier.

A linked queue is more complicated than a stack because it must support fast access to both the head and the tail. To do this, it maintains separate head and tail pointers. Two pointers refer to the node at the tail: the `next` pointer of the current last element, and the tail pointer. To insert a new element successfully, both of these pointers must be updated—atomically. At first glance, this cannot be done with atomic variables; separate CAS operations are required to update the two pointers, and if the first

succeeds but the second one fails the queue is left in an inconsistent state. And, even if both operations succeed, another thread could try to access the queue between the first and the second. Building a nonblocking algorithm for a linked queue requires a plan for both these situations.

Listing 15.6. Nonblocking Stack Using Treiber's Algorithm ([Treiber, 1986](#)).

```
@ThreadSafe
public class ConcurrentStack <E> {
    AtomicReference<Node<E>> top = new AtomicReference<Node<E>>();

    public void push(E item) {
        Node<E> newHead = new Node<E>(item);
        Node<E> oldHead;
        do {
            oldHead = top.get();
            newHead.next = oldHead;
        } while (!top.compareAndSet(oldHead, newHead));
    }

    public E pop() {
        Node<E> oldHead;
        Node<E> newHead;
        do {
            oldHead = top.get();
            if (oldHead == null)
                return null;
            newHead = oldHead.next;
        } while (!top.compareAndSet(oldHead, newHead));
        return oldHead.item;
    }

    private static class Node <E> {
        public final E item;
        public Node<E> next;

        public Node(E item) {
            this.item = item;
        }
    }
}
```

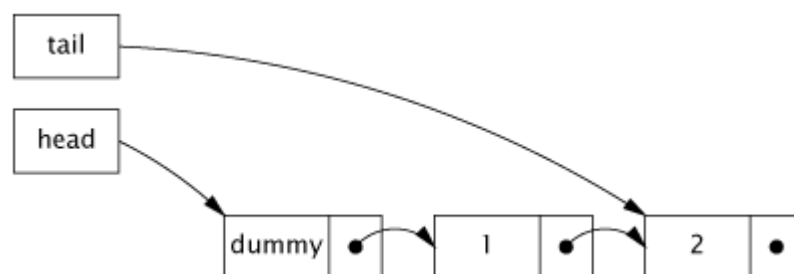
We need several tricks to develop this plan. The first is to ensure that the data structure is always in a consistent state, even in the middle of an multi-step update. That way, if thread *A* is in the middle of a update when thread *B* arrives on the scene, *B* can tell that an operation has been partially completed and knows not to try immediately to apply its own up-

date. Then *B* can wait (by repeatedly examining the queue state) until *A* finishes, so that the two don't get in each other's way.

While this trick by itself would suffice to let threads “take turns” accessing the data structure without corrupting it, if one thread failed in the middle of an update, no thread would be able to access the queue at all. To make the algorithm nonblocking, we must ensure that the failure of a thread does not prevent other threads from making progress. Thus, the second trick is to make sure that if *B* arrives to find the data structure in the middle of an update by *A*, enough information is already embodied in the data structure for *B* to *finish the update for A*. If *B* “helps” *A* by finishing *A*'s operation, *B* can proceed with its own operation without waiting for *A*. When *A* gets around to finishing its operation, it will find that *B* already did the job for it.

`LinkedList` in [Listing 15.7](#) shows the insertion portion of the Michael-Scott nonblocking linked-queue algorithm ([Michael and Scott, 1996](#)), which is used by `ConcurrentLinkedList`. As in many queue algorithms, an empty queue consists of a “sentinel” or “dummy” node, and the head and tail pointers are initialized to refer to the sentinel. The tail pointer always refers to the sentinel (if the queue is empty), the last element in the queue, or (in the case that an operation is in mid-update) the second-to-last element. [Figure 15.3](#) illustrates a queue with two elements in the normal, or *quiescent*, state.

Figure 15.3. Queue with Two Elements in Quiescent State.



Inserting a new element involves updating two pointers. The first links the new node to the end of the list by updating the `next` pointer of the current last element; the second swings the tail pointer around to point to the new last element. Between these two operations, the queue is in the

intermediate state, shown in [Figure 15.4](#). After the second update, the queue is again in the quiescent state, shown in [Figure 15.5](#).

Figure 15.4. Queue in Intermediate State During Insertion.

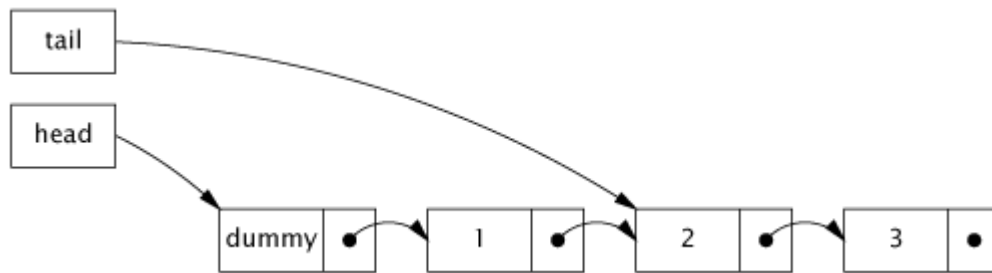
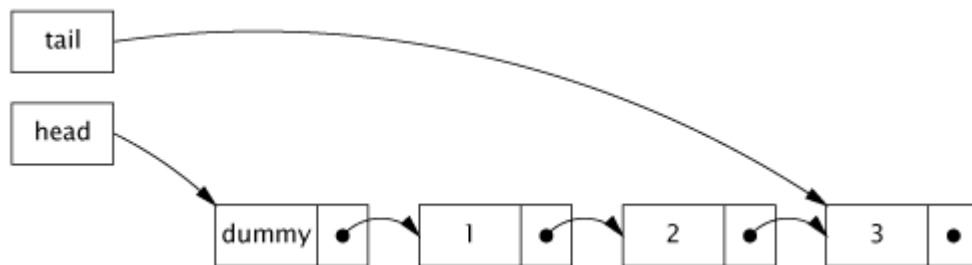


Figure 15.5. Queue Again in Quiescent State After Insertion is Complete.



The key observation that enables both of the required tricks is that if the queue is in the quiescent state, the `next` field of the link node pointed to by `tail` is null, and if it is in the intermediate state, `tail.next` is non-null. So any thread can immediately tell the state of the queue by examining `tail.next`. Further, if the queue is in the intermediate state, it can be restored to the quiescent state by advancing the tail pointer forward one node, finishing the operation for whichever thread is in the middle of inserting an element.^[7]

[7] For a full account of the correctness of this algorithm, see ([Michael and Scott, 1996](#)) or ([Herlihy and Shavit, 2006](#)).

`LinkedList.put` first checks to see if the queue is in the intermediate state before attempting to insert a new element (step A). If it is, then some other thread is already in the process of inserting an element (between its steps C and D). Rather than wait for that thread to finish, the current thread helps it by finishing the operation for it, advancing the tail pointer (step B). It then repeats this check in case another thread has started in-

serting a new element, advancing the tail pointer until it finds the queue in the quiescent state so it can begin its own insertion.

The CAS at step *C*, which links the new node at the tail of the queue, could fail if two threads try to insert an element at the same time. In that case, no harm is done: no changes have been made, and the current thread can just reload the tail pointer and try again. Once *C* succeeds, the insertion is considered to have taken effect; the second CAS (step *D*) is considered “cleanup”, since it can be performed either by the inserting thread or by any other thread. If *D* fails, the inserting thread returns anyway rather than retrying the CAS, because no retry is needed—another thread has already finished the job in its step *B*! This works because before any thread tries to link a new node into the queue, it first checks to see if the queue needs cleaning up by checking if `tail.next` is non-null. If it is, it advances the tail pointer first (perhaps multiple times) until the queue is in the quiescent state.

Listing 15.7. Insertion in the Michael-Scott Nonblocking Queue Algorithm ([Michael and Scott, 1996](#)).


```

@ThreadSafe
public class LinkedQueue <E> {
    private static class Node <E> {
        final E item;
        final AtomicReference<Node<E>> next;

        public Node(E item, Node<E> next) {
            this.item = item;
            this.next = new AtomicReference<Node<E>>(next);
        }
    }

    private final Node<E> dummy = new Node<E>(null, null);
    private final AtomicReference<Node<E>> head
        = new AtomicReference<Node<E>>(dummy);
    private final AtomicReference<Node<E>> tail
        = new AtomicReference<Node<E>>(dummy);

    public boolean put(E item) {
        Node<E> newNode = new Node<E>(item, null);
        while (true) {
            Node<E> curTail = tail.get();
            Node<E> tailNext = curTail.next.get();
            if (curTail == tail.get()) {
                if (tailNext != null) {
                    // Queue in intermediate state, advance tail
                    tail.compareAndSet(curTail, tailNext);
                } else {
                    // In quiescent state, try inserting new node
                    if (curTail.next.compareAndSet(null, newNode)) {
                        // Insertion succeeded, try advancing tail
                        tail.compareAndSet(curTail, newNode);
                        return true;
                    }
                }
            }
        }
    }
}

```

15.4.3. Atomic Field Updaters

Listing 15.7 illustrates the algorithm used by `ConcurrentLinkedQueue`, but the actual implementation is a bit different. Instead of representing each `Node` with an atomic reference, `ConcurrentLinkedQueue` uses an ordinary volatile reference and updates it through the reflection-based `AtomicReferenceFieldUpdater`, as shown in **Listing 15.8**.

Listing 15.8. Using Atomic Field Updaters in `ConcurrentLinkedQueue`.

```
private class Node<E> {
    private final E item;
    private volatile Node<E> next;

    public Node(E item) {
        this.item = item;
    }
}

private static AtomicReferenceFieldUpdater<Node, Node> nextUpdater
    = AtomicReferenceFieldUpdater.newUpdater(
        Node.class, Node.class, "next");
```

The atomic field updater classes (available in `Integer`, `Long`, and `Reference` versions) represent a reflection-based “view” of an existing volatile field so that CAS can be used on existing volatile fields. The updater classes have no constructors; to create one, you call the `newUpdater` factory method, specifying the class and field name. The field updater classes are not tied to a specific instance; one can be used to update the target field for any instance of the target class. The atomicity guarantees for the updater classes are weaker than for the regular atomic classes because you cannot guarantee that the underlying fields will not be modified directly—the `compareAndSet` and arithmetic methods guarantee atomicity only with respect to other threads using the atomic field updater methods.

In `ConcurrentLinkedQueue`, updates to the `next` field of a `Node` are applied using the `compareAndSet` method of `nextUpdater`. This somewhat circuitous approach is used entirely for performance reasons. For frequently allocated, short-lived objects like queue link nodes, eliminating the creation of an `AtomicReference` for each `Node` is significant enough to reduce the cost of insertion operations. However, in nearly all situations, ordinary atomic variables perform just fine—in only a few cases will the atomic field updaters be needed. (The atomic field updaters are also useful when you want to perform atomic updates while preserving the serialized form of an existing class.)

15.4.4. The ABA Problem

The *ABA problem* is an anomaly that can arise from the naive use of compare-and-swap in algorithms where nodes can be recycled (primarily in

environments without garbage collection). A CAS effectively asks “Is the value of V still A ?”, and proceeds with the update if so. In most situations, including the examples presented in this chapter, this is entirely sufficient. However, sometimes we really want to ask “Has the value of V changed since I last observed it to be A ?” For some algorithms, changing V from A to B and then back to A still counts as a change that requires us to retry some algorithmic step.

This *ABA problem* can arise in algorithms that do their own memory management for link node objects. In this case, that the head of a list still refers to a previously observed node is not enough to imply that the contents of the list have not changed. If you cannot avoid the ABA problem by letting the garbage collector manage link nodes for you, there is still a relatively simple solution: instead of updating the value of a reference, update a *pair* of values, a reference and a version number. Even if the value changes from A to B and back to A , the version numbers will be different. `AtomicStampedReference` (and its cousin `AtomicMarkableReference`) provide atomic conditional update on a pair of variables. `AtomicStampedReference` updates an object reference-integer pair, allowing “versioned” references that are immune^[8] to the ABA problem. Similarly, `AtomicMarkableReference` updates an object reference-boolean pair that is used by some algorithms to let a node remain in a list while being marked as deleted.^[9]

^[8] In practice, anyway; theoretically the counter could wrap.

^[9] Many processors provide a double-wide CAS (CAS2 or CASX) operation that can operate on a pointer-integer pair, which would make this operation reasonably efficient. As of Java 6, `AtomicStampedReference` does not use double-wide CAS even on platforms that support it. (Double-wide CAS differs from DCAS, which operates on two unrelated memory locations; as of this writing, no current processor implements DCAS.)

Summary

Nonblocking algorithms maintain thread safety by using low-level concurrency primitives such as compare-and-swap instead of locks. These low-level primitives are exposed through the atomic variable classes,

which can also be used as “better volatile variables” providing atomic update operations for integers and object references.

Nonblocking algorithms are difficult to design and implement, but can offer better scalability under typical conditions and greater resistance to liveness failures. Many of the advances in concurrent performance from one JVM version to the next come from the use of nonblocking algorithms, both within the JVM and in the platform libraries.